Bharath Kumar Nersu
2023001411

# AI Subsystem Integration Planning & Microservice Architecture

Development Branch: feature/ai-workflow:

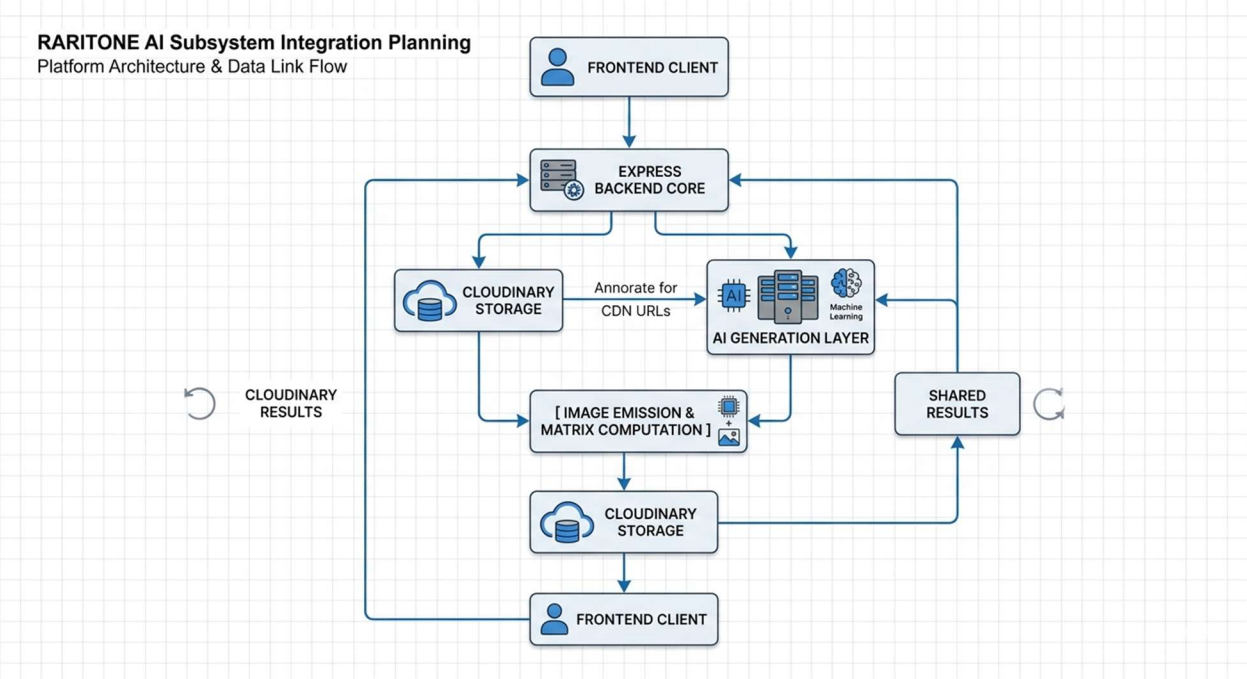
## 1. Executive Summary

This document provides the architectural design, routing definitions, and component communication flows required to introduce AI capabilities—specifically Virtual Try-On pipelines and user Recommendation Engines—into the existing Raritone Node.js/Express ecosystem.

This plan leverages the platform's existing media infrastructure, utilizing asynchronous tracking identifiers to handle long-running generative model operations without degrading API throughput.

## 2. Platform Architecture & Data Link Flow

The integration layout maps out a clear separation of concerns between client requests, core backend logic, cloud asset storage, and isolated deep-learning models.



Bharath Kumar Nersu
2023001411

## 3. Core Workflow Mechanisms

### A. Asynchronous Try-On Result Flow

Because deep-learning image generation models (such as virtual try-on diffusion pipelines) exhibit higher computational latency than standard database lookups, an asynchronous request-reply engine model is used:

1. **Ingestion & Validation:** The client submits a payload containing a targeted product ID and a baseline model image. The backend route utilizes the protect middleware to guarantee session security.
2. **Asset Staging:** The raw assets are processed via specialized storage buckets on Cloudinary, converting multi-part files into highly available CDN addresses.
3. **Task Deferral:** The backend drops a processing token to the AI generation cluster containing the image array addresses and immediately returns a 202 Accepted status along with a unique sessionId to the frontend client. This frees up the Express application thread.
4. **Status Polling & Settlement:** The frontend application routinely requests execution states via GET /api/tryOnRoutes/tryon/:id. Once the generation engine pushes the final composite asset back to Cloudinary, the status updates to completed, exposing the generated content link directly to the layout view engine.

### B. Intelligent Recommendation Engine Workflow

1. **Behavioral Telemetry Aggregation:** The application captures underlying interaction variables generated within the application environment, referencing data structures updated via the Wardrobe database module and user-driven state changes inside individual Wishlist schemas.
2. **Vector Space Profiling:** Processing scripts match categorized metadata arrays (such as fabric matrices, color codes, and apparel category classifications) to construct isolated customer preference tensors.
3. **Similarity Index Lookup:** The microservice executes multi-dimensional similarity algorithms across current product database inventories, identifying high-affinity pairs and cross-category clothing combinations.

4. **Optimized Structural Feed Return:** Data blocks are delivered down to the client interface dynamically, refreshing personalized landing frames without initiating deep layout re-renders.

## 4. Final Production API Structural Reference

**Route 1: Initiate Asynchronous Virtual Try-On**

- **Path Reference:** POST /api/tryOnRoutes/tryon
- **Access Tier:** Private (Enforced via protect Authorization Middleware)
- **Content-Type:** application/json

**Production Request Payload Structure:**

```
JSON

{
  "apparelId": "prod_665544",
  "userImageId": "cloudinary_raw_user_reference_01",
  "config": {
    "fit": "regular",
    "size": "L"
  }
}
```

Production Response Blueprint (202 Accepted):

```
JSON

{
  "success": true,
  "message": "Virtual try-on AI workflow initiated successfully.",
  "sessionId": "try_mock_session_12345",
  "status": "processing",
  "estimatedTimeSeconds": 15
}
```

**Route 2: Query Try-On Operation Matrix Result**

- **Path Reference:** GET /api/tryOnRoutes/tryon/:id
- **Access Tier:** Private (Enforced via protect Authorization Middleware)

**Production Response Blueprint (200 OK - Processing Finalized):**

```JSON
{
  "success": true,
  "sessionId": "try_mock_session_12345",
  "status": "completed",
  "result": {
    "originalUserImage": "https://res.cloudinary.com/raritone/image/upload/sampl
    "apparelImage": "https://res.cloudinary.com/raritone/image/upload/sample_pro
    "outputImage": "https://res.cloudinary.com/raritone/image/upload/generated_t
  }
}
```

**5. Summary of Dev Changes (Git Trace)**

To document completion of the planning milestone, the following changes were successfully introduced, staged, and pushed to the origin repository:

- **Modified File:** routes/tryOnRoutes.js
    - Preserved legacy core handlers (uploadTryOn, getTryOnHistory, deleteTryOn).
    - Appended planned structural endpoints mapping POST /tryon and GET /tryon/:id using Express Router architecture.
- **Target Integration Repository Branch:** feature/ai-workflow